

ME 2261 - Embedded Systems Log BookTask 1

Input character in virtual terminal while LED is blinking in PORT C.
The output for ASCII value of character will turn on at PORT B.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

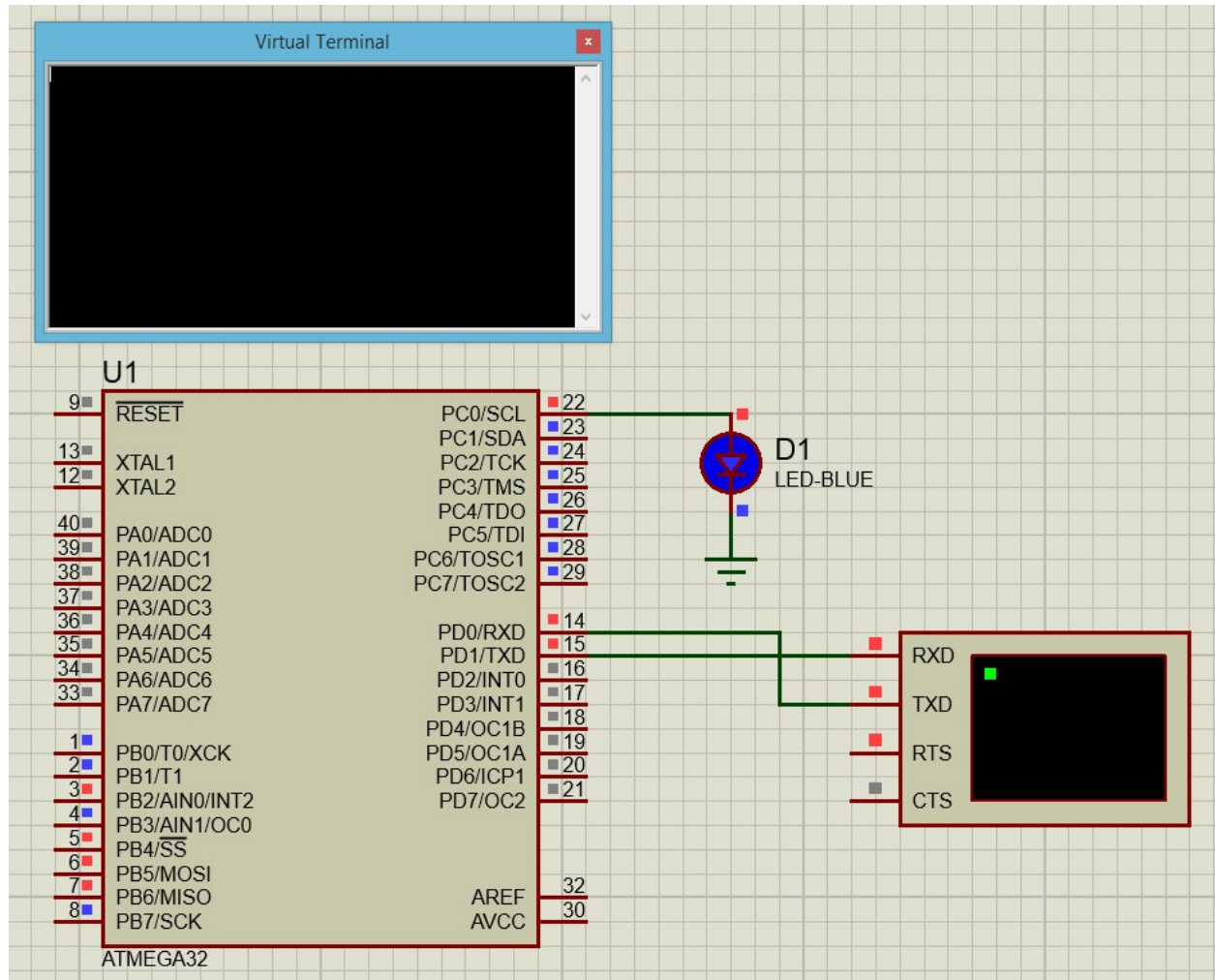
void USART_init(void) {
    UCSRA = 0x00;
    UCSRB = 0x98;
    UCSRC = 0x86;
    UBRRL = 0x33;
    UBRRH = 0x00;
}

ISR(USART_RXC_vect) {
    PORTB = UDR;
}

int main(void) {
    USART_init();
    sei();
    DDRB = 0xFF;
    DDRC = 0xFF;

    while (1) {
        PORTC |= (1 << PC0);
        _delay_ms(2000);

        PORTC &= ~(1 << PC0);
        _delay_ms(2000);
    }
    return 0;
}
```



Task 2

There will be a delay in the same task 1 here when we interrupt the same task like this.

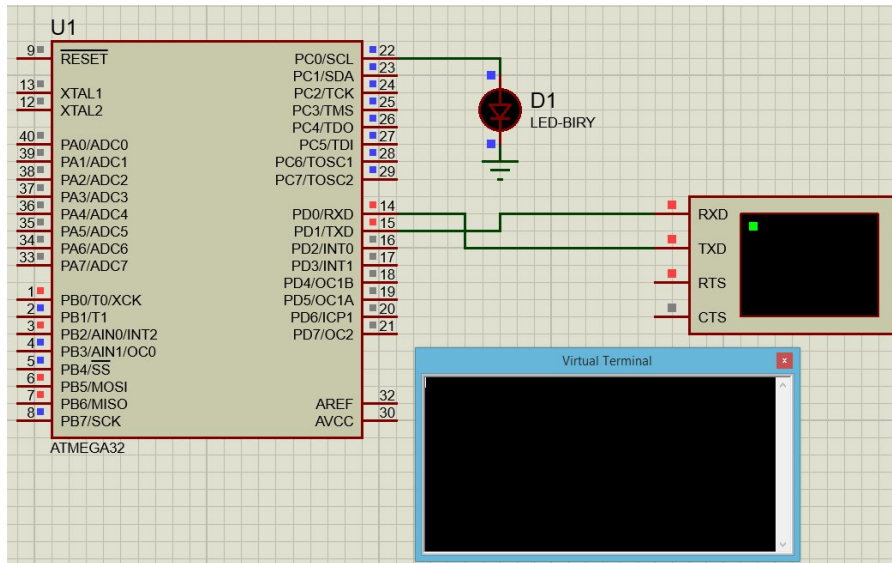
```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

void USART_init(void) {
    UCSRA = 0x00;
    UCSRB = 0b00011000;
    UCSRC = 0x86;
    UBRRL = 0x33;
    UBRRH = 0x00;
}

ISR(USART_RXC_vect) {
    PORTB = UDR;
}

int main(void) {
    USART_init();
    sei();
    DDRB = 0xFF;
    DDRC = 0xff;

    while (1) {
        while((UCSRA & (1<<RXC)) == 0x00){
            PORTC = 0x00000010;
            _delay_ms(1000);
            PORTC = 0x00000000;
            _delay_ms(1000);
        }
        PORTB = UDR;
        _delay_ms(1000);
    }
    return 0;
}
```



Task 3

When the push button is pressed or released, the PORTB becomes $\sim$ PORTB.

```
#define F_CPU 8000000UL
```

```
#include <avr/io.h>
```

```
#include <avr/interrupt.h>
```

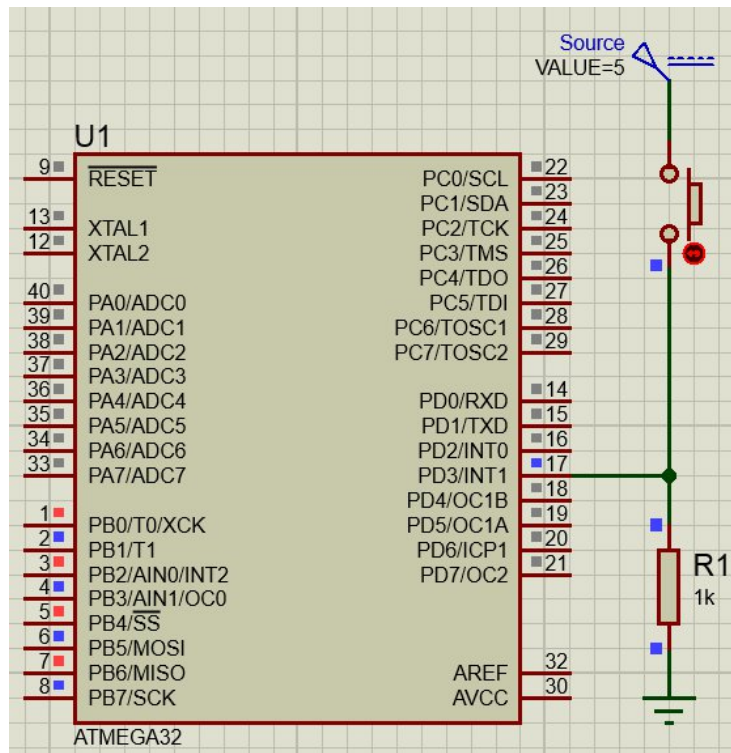
```
#include <util/delay.h>
```

```
ISR (INT1_vect)
```

```
{
    PORTB = (~PORTB);
}
```

```
int main(void)
```

```
{
    GICR = (1<<INT1);
    MCUCR = (1<<ISC10);
    sei();
    DDRB = 0xFF;
    DDRC = 0X0;
    PORTB = 0b01010101;
    while (1){;}
    return 0;
}
```



Task 4

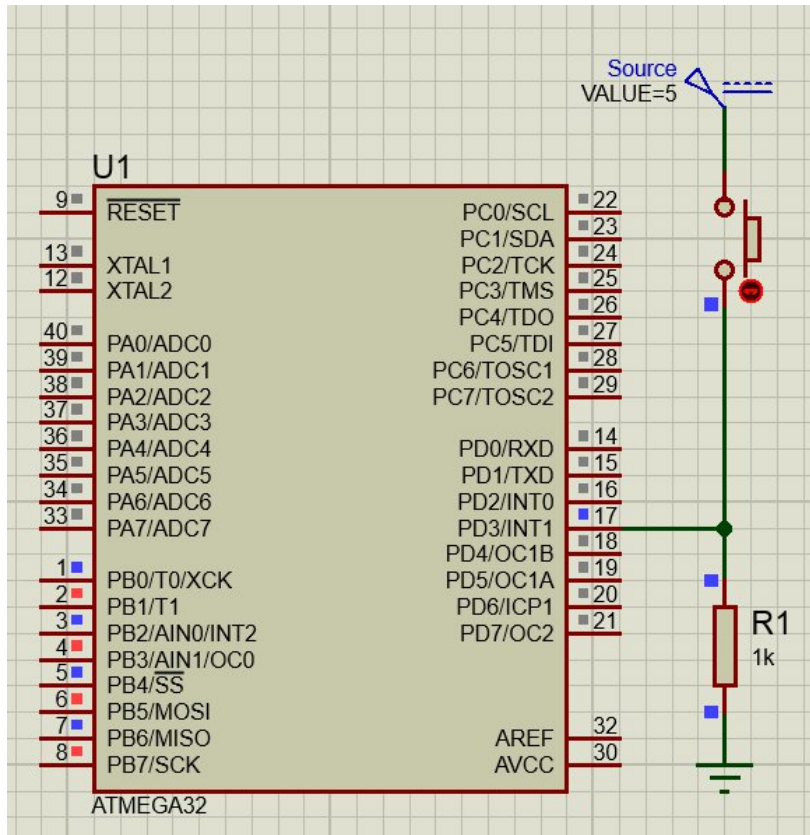
When the push button is pressed & released, the PORTB becomes $\sim$ PORTB.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
```

```
ISR (INT1_vect)
{
    PORTB = (~PORTB);
}
```

```
int main(void)
{
    GICR = (1<<INT1);
    MCUCR = (1<<ISC10) | (1<<ISC11);
    sei();
    DDRB = 0xFF;
    DDRC = 0X0;
    PORTB = 0b01010101;
    while (1){;}
    return 0;
}
```

}



Task 5

Without prescaler

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

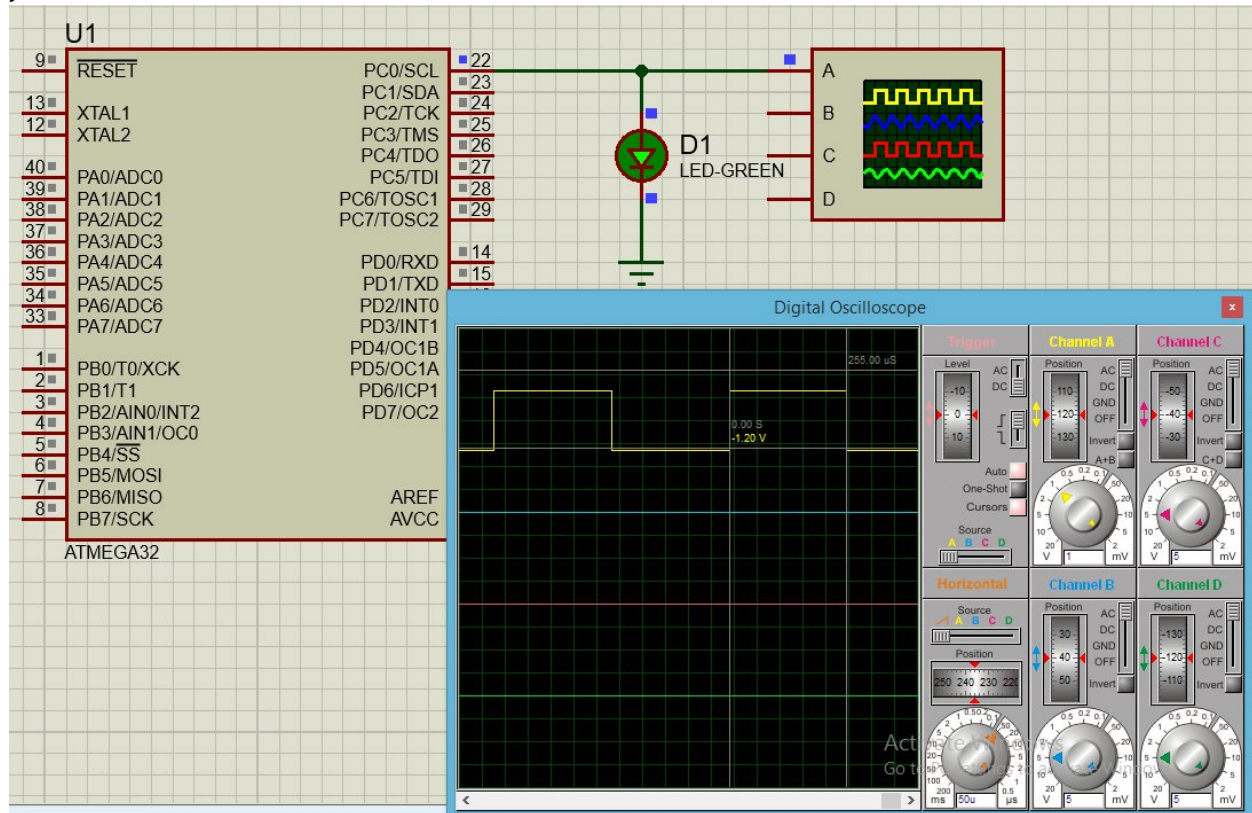
void timer0_init()
{
    TCCR0 |= (1 << CS00);
    TCNT0 = 0;
}

int main(void){
    DDRC |= (1 << 0);
    timer0_init();
    while(1){
        if (TCNT0 >= 249)
        {
```

```

    PORTC ^= (1<<0);
    TCNT0 = 0;
}
}
}

```



Task 6

Using prescaler

```

#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

void timer0_init()
{
    TCCR0 |= (1 << CS02); // prescaler 256
    TCNT0 = 0;
}

int main(void){
    DDRC |= (1 << 0);

```



```

timer0_init();
while(1){
    if (TCNT0 >= 249)
    {
        PORTC ^= (1<<0);
        TCNT0 = 0;
    }
}

```

